Day 133 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 20, Day 1 reference

Watch: (ML 1.5) Generative vs discriminative models (mathematicalmonk) | IAML2.23: Generative vs. discriminative learning (Victor Lavrenko)

`day133_generative_vs_discriminative.py`, segment by segment, in the order `main()` uses things. The Gaussian class models (fit_gaussians, log_gauss, log_joint, posterior, log_px, lda_linear, sample_class) and the learning curve are new; logistic regression is Day 116's fit_softmax, PCA is Day 123's, ROC AUC is Day 117's, all imported unchanged. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: X = rows of features · mu = class means · S, cov = covariances · L = a Cholesky factor ($L\,L^{T} = S$) · keep = the features that are seen · J = log p(x, class)

Segment 0 — Setup

Imports

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
for w in ("week-19-unsupervised-learning-ii", "week-18-unsupervised-learning-i",
    "week-17-classical-ml-kernels-ensembles"):
    sys.path.insert(0, os.path.join(HERE, "..", w))
with contextlib.redirect_stdout(io.StringIO()):
    import day126_autoencoders as d126m
    import day123_exercise as d123
    import day117_exercise as d117
    import day116_exercise as d116
```

- `for w in (...): sys.path.insert(...)` — three earlier weeks' folders, so their exercise files and Day 126's loader can be imported.
- `with contextlib.redirect_stdout(io.StringIO()):` — the imported lessons print their own checks; this hides that output.

Segment 1 — The generative model

fit_gaussians: three kinds of class model

MATH

$$\Sigma_{shared} = (1/n)\,\Sigma_i\,(x_i - \mu_{y_i})(x_i - \mu_{y_i})^T + reg\ I$$

CODE

```
def fit_gaussians(X, y, k, kind="shared", reg=1e-3):
    d = X.shape[1]; mu = np.array([X[y == c].mean(0) for c in range(k)])
    prior = np.bincount(y, minlength=k) / len(y)
    if kind == "shared":
        D = X - mu[y]; S = D.T @ D / len(X) + reg * np.eye(d)
        cov = np.repeat(S[None], k, 0)
    elif kind == "diagonal":
        cov = np.array([np.diag(X[y == c].var(0) + reg) for c in range(k)])
    else:
        cov = np.array([np.cov(X[y == c].T, bias=True).reshape(d, d) + reg * np.eye(d) for c in
            range(k)])
    return {"mu": mu, "cov": cov, "prior": prior}
```

- `np.bincount(y, minlength=k) / len(y)` — each class's share of the training rows: p(class).
- `D = X - mu[y]` — fancy indexing: every row minus its own class's mean, in one line.
- `np.repeat(S[None], k, 0)` — the shared covariance copied once per class, so every kind has the same shape (k, d, d).
- `np.diag(X[y == c].var(0) + reg)` — naive Bayes: a covariance with only the diagonal (each feature's variance) filled.
- `np.cov(X[y == c].T, bias=True)` — np.cov wants features in rows, hence the .T; bias=True divides by n, like the shared case.

log_gauss and log_joint

MATH

$$\log N(x;\,\mu,\,S) = -\tfrac{1}{2}\,[\ |L^{-1}(x-\mu)|^2 + \log\det S + d\log 2\pi\ ]$$

CODE

```
def log_gauss(X, mu, S):
    L = np.linalg.cholesky(S); Z = np.linalg.solve(L, (X - mu).T)
    return -0.5 * ((Z ** 2).sum(0) + 2 * np.log(np.diag(L)).sum() + len(mu) * np.log(2 * np.pi))
def log_joint(model, X, keep=None):
    idx = np.arange(X.shape[1]) if keep is None else keep
    return np.stack([log_gauss(X[:, idx], m[idx], S[np.ix_(idx, idx)]) + np.log(p)
                     for m, S, p in zip(model["mu"], model["cov"], model["prior"])], 1)
```

- `np.linalg.cholesky(S)` — a lower-triangular L with $L\,L^T = S$: cheaper and safer than inverting S.
- `np.linalg.solve(L, (X - mu).T)` — $Z = L^{-1}(x - \mu)$ for every row at once; $|Z|^2$ is the Mahalanobis distance.
- `2 * np.log(np.diag(L)).sum()` — log det S, from the diagonal of L, with no overflow.
- `S[np.ix_(idx, idx)]` — the rows and columns of the seen features: the marginal's covariance.
- `np.stack([...], 1)` — one column per class.

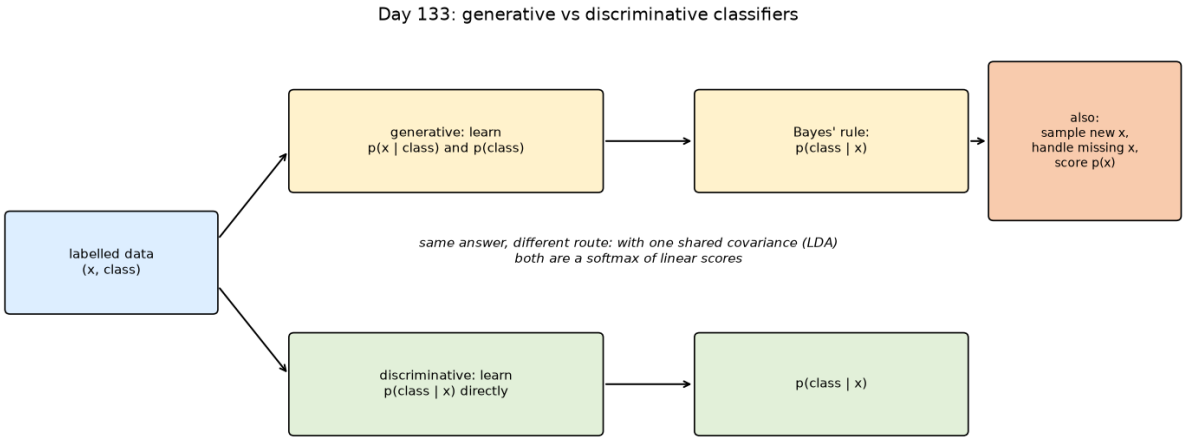
posterior, log_px, lda_linear, sample_class

CODE

```
def posterior(model, X, keep=None):
    J = log_joint(model, X, keep); J -= J.max(1, keepdims=True); P = np.exp(J)
    return P / P.sum(1, keepdims=True)
def log_px(model, X):
    J = log_joint(model, X); m = J.max(1)
    return m + np.log(np.exp(J - m[:, None]).sum(1))
def lda_linear(model):
    Si = np.linalg.inv(model["cov"][0]); W = Si @ model["mu"].T
    b = -0.5 * np.einsum("cd,dc->c", model["mu"], W) + np.log(model["prior"])
    return W, b
def sample_class(model, c, n, rng):
    L = np.linalg.cholesky(model["cov"][c])
    return model["mu"][c] + rng.standard_normal((n, len(L))) @ L.T
```

- `J -= J.max(1, keepdims=True)` — subtracting each row's largest log value before exp: the log-sum-exp trick against overflow.
- `m + np.log(np.exp(J - m[:, None]).sum(1))` — log p(x) = log of the sum of p(x, class), computed the same stable way.
- `np.einsum("cd,dc->c", mu, W)` — for each class c, the dot product of mu_c with column c of W.
- `rng.standard_normal((n, len(L))) @ L.T` — n draws of N(0, I), each turned by L into the class's covariance.

SYSTEM DIAGRAM



Two routes to p(class | x).

Segment 2 — The discriminative model and the experiments

fit_logistic and learning_curve

CODE

```
def fit_logistic(X, y, k, lam=1e-3, steps=500):
    mu, sd = X.mean(0), np.maximum(X.std(0), 0.05)
    W, b = d116.fit_softmax((X - mu) / sd, y, k, lam=lam, steps=steps)
    return lambda Z: d116.softmax(((Z - mu) / sd) @ W + b)
def learning_curve(fits, X, y, Xte, yte, sizes, draws, seed=0):
    rng = np.random.RandomState(seed); k = int(y.max()) + 1
    err = {m: np.zeros((len(sizes), draws)) for m in fits}
    for i, n in enumerate(sizes):
        for r in range(draws):
            while True:
                tr = rng.choice(len(X), n, replace=False)
                if np.bincount(y[tr], minlength=k).min() >= 2:
                    break
            for m, fit in fits.items():
                err[m][i, r] = np.mean(fit(X[tr], y[tr], k)(Xte).argmax(1) != yte)
    return err
```

- `np.maximum(X.std(0), 0.05)` — a floor on each feature's spread: a pixel that never varied in training (spread 0) would otherwise be multiplied enormously when a new image has ink there.
- `return lambda Z: ...` — a closure: the fitted classifier, remembering mu, sd, W and b.
- `while True: ... break` — redraw the random subset until every class has at least two rows (a class covariance needs them).
- `fit(X[tr], y[tr], k)(Xte)` — fit, then call the returned function on the test rows, in one expression.

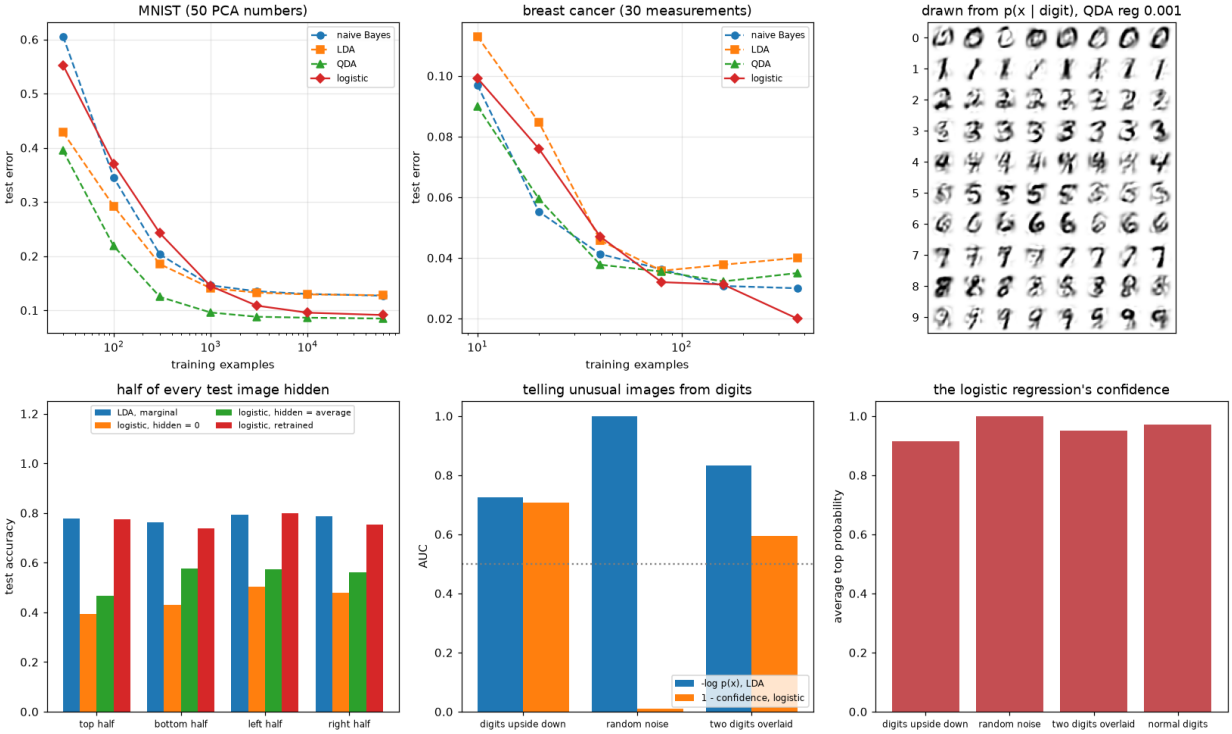
demo_missing: four patterns of hidden pixels

CODE

```
te = np.arange(60000, 70000); g = np.arange(784).reshape(28, 28)
halves = {"top half seen": g[:14].ravel(), "bottom half seen": g[14:].ravel(), "left half seen":
    g[:, :14].ravel(), "right half seen": g[:, 14:].ravel()}
mean = X[:60000].mean(0)
print(f"   {'pattern':18s}  LDA, hidden pixels left out  logistic, hidden = 0  logistic, hidden =
    average  logistic retrained on that half")
rows = {}
for name, keep in halves.items():
    a = np.mean(posterior(lda, X[te], keep).argmax(1) == y[te])
    Z0 = X[te].copy(); hid = np.setdiff1d(np.arange(784), keep); Z0[:, hid] = 0
    Zm = X[te].copy(); Zm[:, hid] = mean[hid]
    b, c = np.mean(lr(Z0).argmax(1) == y[te]), np.mean(lr(Zm).argmax(1) == y[te])
    d = np.mean(fit_logistic(X[:60000][:, keep], y[:60000], 10)(X[te][:, keep]).argmax(1) ==
        y[te])
```

- `g[:14].ravel(), g[:, :14].ravel()` — pixel numbers of the top rows or the left columns, flattened to a list.
- `np.setdiff1d(np.arange(784), keep)` — every pixel not kept: the hidden ones.
- `Z0[:, hid] = 0` — fill the hidden pixels of every image at once.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.linalg.cholesky(S)` — the lower-triangular square root of a covariance.
- `np.linalg.solve(L, B)` — solve L Z = B without forming an inverse.
- `S[np.ix_(idx, idx)]` — a sub-matrix: chosen rows and the same columns.
- `np.einsum("cd,dc->c", A, B)` — a sum over d for each c, written as index labels.
- `np.setdiff1d(a, b)` — the elements of a that are not in b.
- `np.bincount(y, minlength=k)` — how many times each class appears.

End of Day 133 syntax reference. For the experiments and what they showed, see the Day 133 complete guide.